

UNIVERSITÀ DI BOLOGNA



School of Engineering
Master Degree in Automation Engineering

Distributed Autonomous Systems
Course Project

Professors:
Giuseppe Notarstefano
Ivano Notarnicola

Students:
Nicolo' Faedo
Mirco Paltrinieri
Luca Sabatini

Academic year 2025/2026

Abstract

This project, developed for the Distributed Autonomous Systems course at the University of Bologna, explores the implementation and evaluation of distributed optimization strategies for multi-robot fleets. The work is divided into two primary tasks: cooperative target localization and aggregative fleet control.

Initially, we implemented a Gradient Tracking algorithm to solve a distributed consensus optimization problem, testing its performance across various communication topologies, such as cycle, path, star and random graphs using Metropolis-Hastings weights. This foundational algorithm was then applied to a cooperative localization scenario, where a team of robots aims to estimate the positions of multiple targets based on noisy distance measurements.

The second phase of the project focuses on Aggregative Optimization. We designed and implemented an Aggregative Tracking algorithm to manage a multi-robot system where individual agents must balance moving toward private targets with the collective requirement of maintaining fleet cohesion. Finally, this aggregative control strategy was ported to a practical ROS2 environment, where a clear visualization of the fleet dynamics is represented on a live RViz2 visualization. The performance of all implemented algorithms is documented through the analysis of cost function evolution and gradient norms across iterations.

Contents

Introduction	4
1 Task 1: Multi-Robot Target Localization	6
1.1 Task 1.1: Distributed Consensus Optimization	6
1.1.1 Gradient Tracking algorithm	6
1.1.2 Results	7
1.2 Task 1.2: Cooperative Multi-Robot Target Localization . . .	13
1.2.1 Gradient Tracking implementation	13
1.2.2 Results	14
2 Task 2: Aggregative Optimization for Multi-Robot Systems	19
2.1 Task 2.1: Distributed Aggregative Optimization	19
2.1.1 Aggregative Tracking algorithm	19
2.1.2 Results	21
2.2 Task 2.2: ROS 2 Implementation	30
2.2.1 Results	31
Conclusions	33
2.3 Conclusion	33

Introduction

Distributed autonomous systems represent a milestone of modern robotics, enabling fleets of agents to solve complex problems that would be infeasible for a single robot. The ability to cooperate, share information, and optimize objectives collectively allows for robust solutions in fields ranging from aerospace to logistics. This project, developed for the Distributed Autonomous Systems course, aims to implement and evaluate distributed optimization strategies to address two specific challenges: cooperative target localization and aggregative fleet control.

The first part of the work focuses on the problem of *Multi-Robot Target Localization*. In this scenario, a fleet of robots must cooperate to estimate the position of target vehicles based on noisy local measurements. The core of this problem lies in reaching a consensus among agents regarding the estimated variables. To achieve this, we employ the Gradient Tracking algorithm to solve a general consensus optimization problem, formally described as:

$$\min_z \sum_{i=1}^N l_i(z)$$

where $z \in \mathbb{R}^d$ represents the global decision vector (the estimated target positions), and $l_i(z)$ is the local cost function available to robot i , representing the error in its observations. This approach ensures that, despite having only partial information, the team converges to a common optimal solution.

The second part of the project addresses *Aggregative Optimization*. Here, the objective shifts from pure estimation to control: agents must optimize their position to track private targets while maintaining fleet cohesion. This is modeled as a minimization problem where the cost depends not only on the agent's state but also on an aggregative variable, $\sigma(z)$, representing the collective behavior of the team (e.g., the barycenter). The dynamics of this problem are governed by:

$$\min_{z \in \mathbb{R}^{dN}} \sum_{i=1}^N l_i(z_i, \sigma(z)) \quad \text{with} \quad \sigma(z) := \frac{1}{N} \sum_{i=1}^N \phi_i(z_i)$$

where z_i is the position of the i -th robot and ϕ_i maps the individual state to the aggregative value.

The report will follow this structure:

chapter 1 details the implementation of the *Gradient Tracking algorithm*. It begins with a preliminary analysis of consensus optimization on various graph topologies (cycle, path, star, random) using Metropolis-Hastings weights. Subsequently, the algorithm is applied to the cooperative localization task, presenting simulation results for the estimation of multiple targets. chapter 2 focuses on the *Aggregative Tracking algorithm*. It discusses the design of cost functions that balance tracking private targets with maintaining fleet tightness. Finally, this chapter presents a practical implementation of the aggregative strategy within a **ROS 2** environment, employing **RViz** to provide a clear and intuitive visualization of the fleet's behavior and the results obtained through simulations.

Chapter 1

Task 1: Multi-Robot Target Localization

1.1 Task 1.1: Distributed Consensus Optimization

1.1.1 Gradient Tracking algorithm

To solve the consensus optimization problem in a distributed manner, we adopt the *Gradient Tracking* algorithm. It is a more advanced solution than a standard distributed gradient descent methods that require diminishing step-sizes to converge to the exact optimal solution, resulting in slow convergence rates. The Gradient Tracking algorithm addresses this limitation by introducing an auxiliary variable that tracks the average gradient of the global cost function, allowing the use of a constant step-size α while achieving linear convergence to the optimal solution z^* .

The algorithm proceeds by maintaining two local variables for each agent i : the solution estimate z_i^k and the gradient tracker s_i^k .

The update laws at iteration k are defined as follows:

$$\begin{aligned} z_i^{k+1} &= \sum_{j \in \mathcal{N}_i} a_{ij} z_j^k - \alpha s_i^k & z_i^0 &\in \mathbb{R}^d \\ s_i^{k+1} &= \sum_{j \in \mathcal{N}_i} a_{ij} s_j^k + \nabla \ell_i(z_i^{k+1}) - \nabla \ell_i(z_i^k) & s_i^0 &= \nabla \ell_i(z_i^0) \end{aligned}$$

where $\mathcal{N}_i$ denotes the set of neighbors of agent i , and a_{ij} are the weights of the communication graph. This mechanism ensures that, asymptotically, the direction s_i^k aligns with the average global gradient, driving all agents toward the global minimizer.

The convergence of the algorithm relies on the adjacency matrix $A = [a_{ij}]$ being doubly stochastic. To satisfy this requirement for various graph topologies (e.g., cycle, star, random), we employ the Metropolis-Hastings method

to determine the weights.

Let d_i and d_j be the degrees of nodes i and j respectively. The weights are computed as:

$$a_{ij} = \begin{cases} \frac{1}{1+\max(d_i, d_j)} & \text{if } j \in \mathcal{N}_i, j \neq i \\ 1 - \sum_{k \in \mathcal{N}_i, k \neq i} a_{ik} & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

This method guarantees that the resulting weight matrix is symmetric and doubly stochastic, provided the underlying graph is connected.

For the initial validation of the algorithm (Task 1.1), we consider a distributed optimization problem characterized by local quadratic cost functions. Specifically, for each agent i , the cost function $\ell_i(z)$ is defined as:

$$\ell_i(z) = \frac{1}{2} z^\top Q_i z + r_i^\top z$$

where $z \in \mathbb{R}^d$ is the decision variable. The parameters $Q_i \in \mathbb{R}^{d \times d}$ and $r_i \in \mathbb{R}^d$ are generated randomly for each agent such that:

- Q_i is a diagonal matrix with positive entries, ensuring the problem is convex.
- r_i is a vector drawn from a normal distribution.

Consequently, the local gradient required for the update step (1.1b) is computed as:

$$\nabla \ell_i(z) = Q_i z + r_i$$

1.1.2 Results

The simulation results clearly demonstrate the effectiveness of the implemented Gradient Tracking algorithm across all tested graph topologies. In every scenario, the total cost function exhibits a monotonic decrease, while the gradient norms exponentially converge toward zero. This behavior confirms that the system achieves stability and that the agents successfully reach consensus on the optimal solution.

Cycle graph

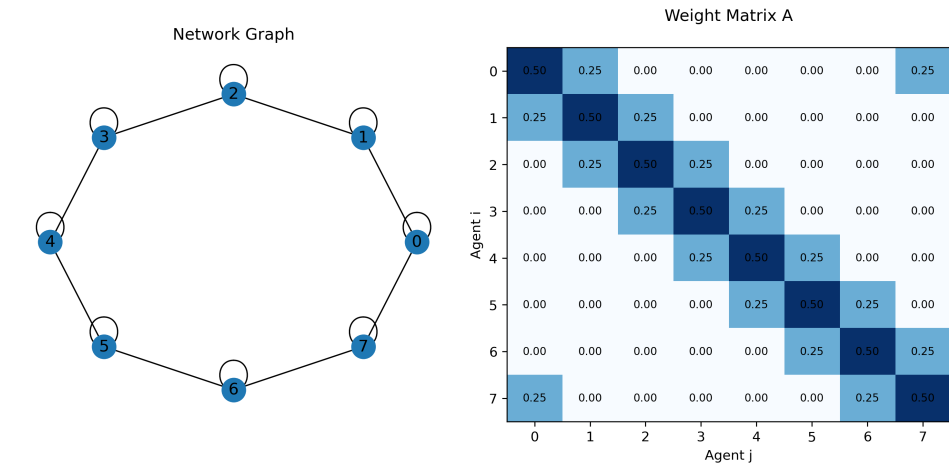


Figure 1.1: Cycle graph Topology

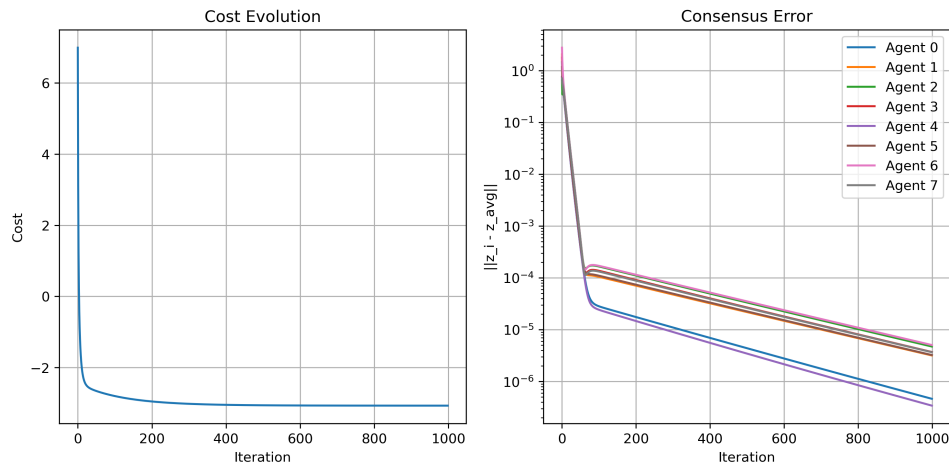


Figure 1.2: Cycle graph Cost consensus

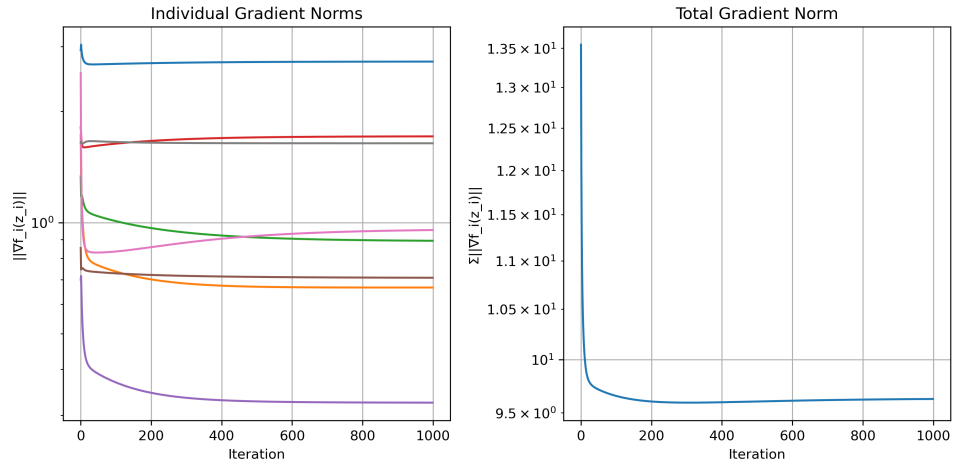


Figure 1.3: Cycle graph Gradient norms

Path graph

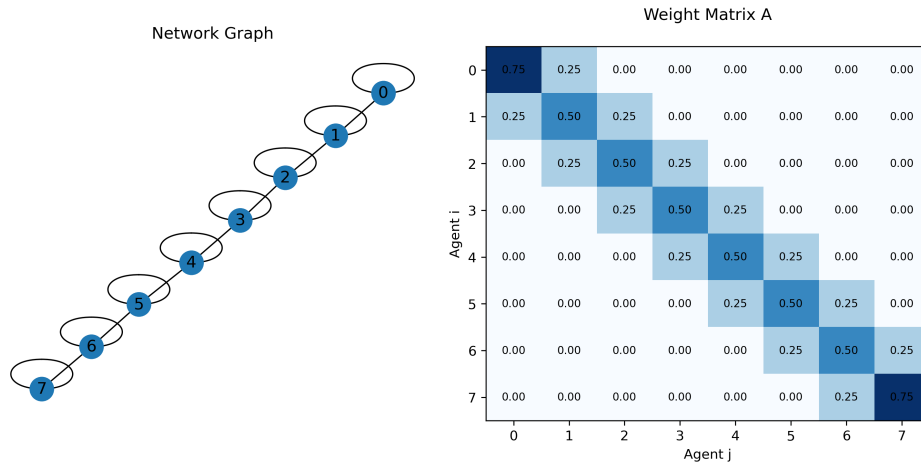


Figure 1.4: Path graph Topology

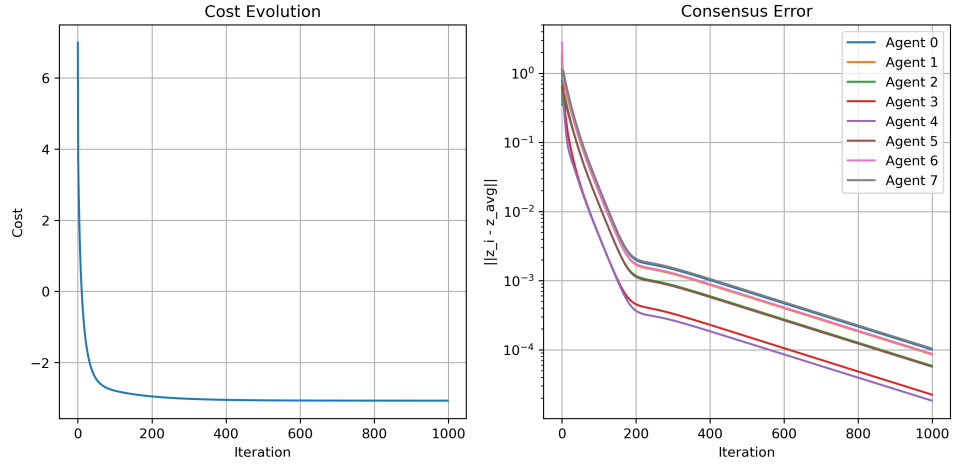


Figure 1.5: Path graph Cost consensus

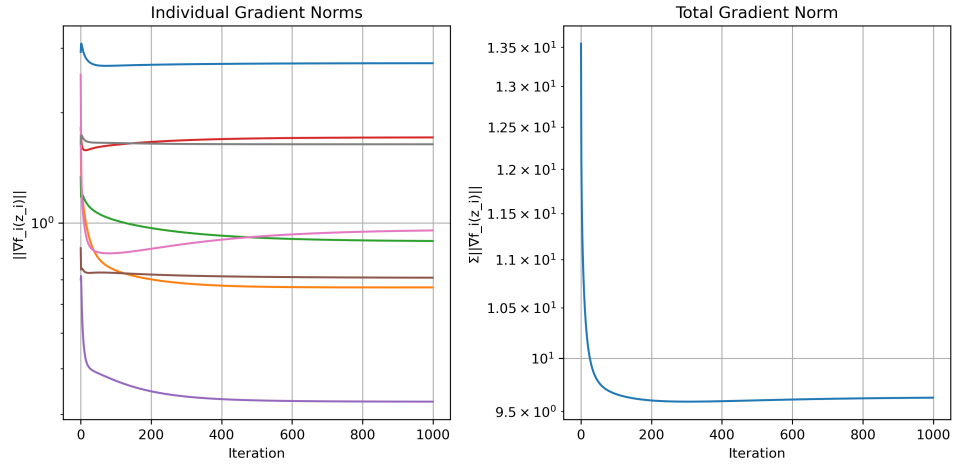


Figure 1.6: Path graph Gradient norms

Random graph

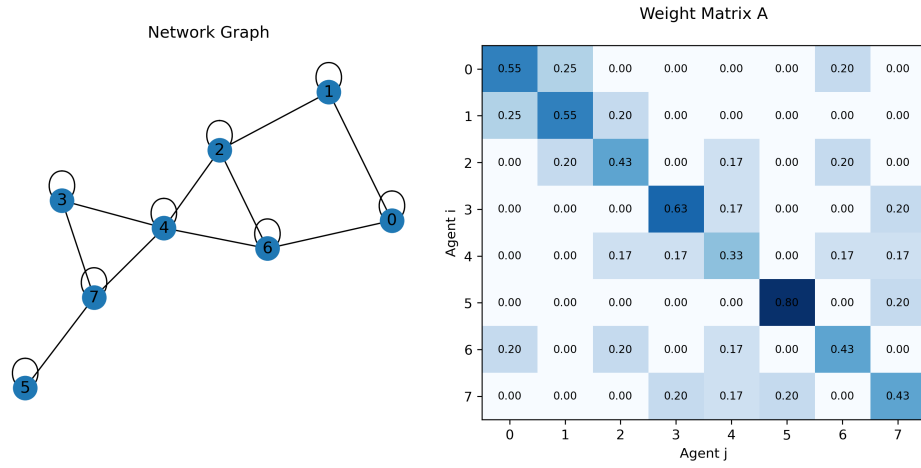


Figure 1.7: Random graph Topology

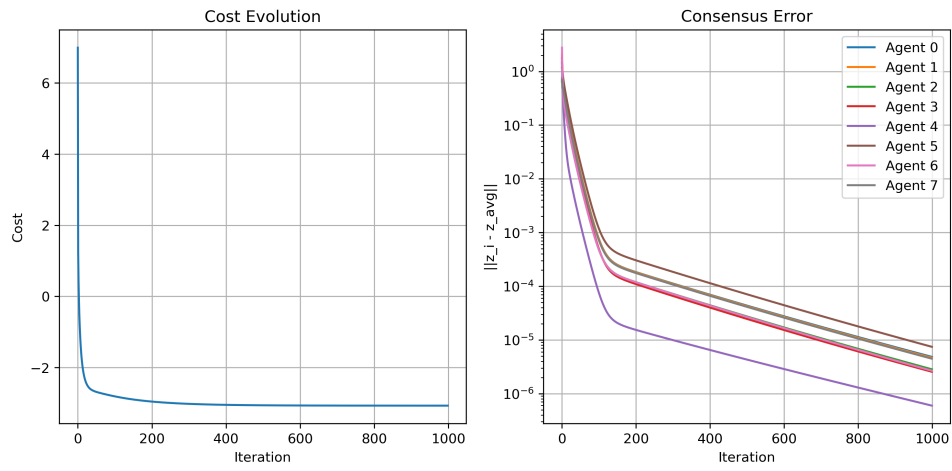


Figure 1.8: Random graph Cost consensus

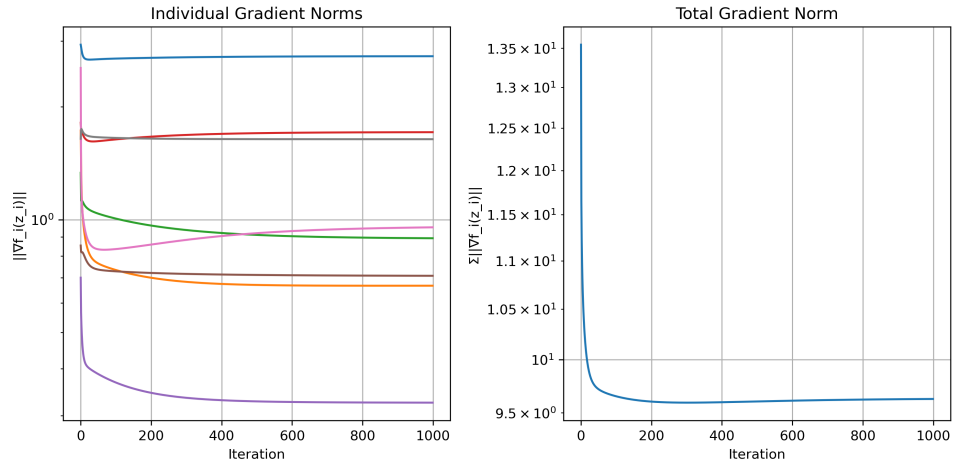


Figure 1.9: Random graph Gradient norms

Star graph

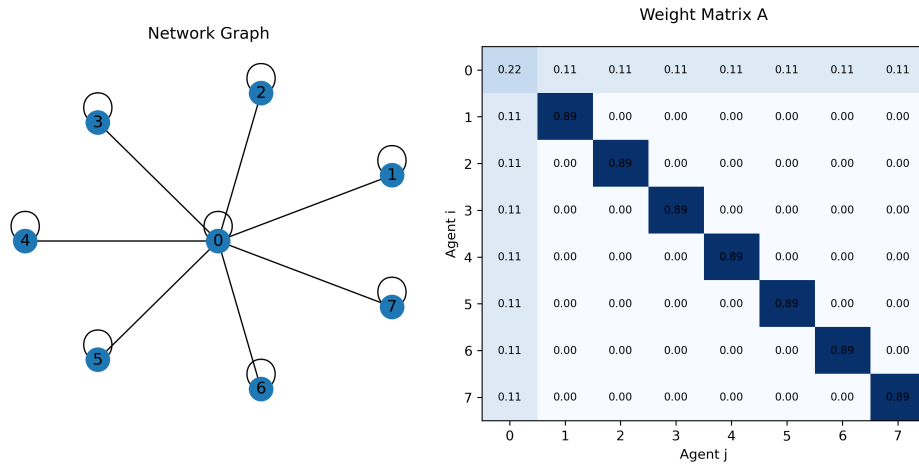


Figure 1.10: Star graph Topology

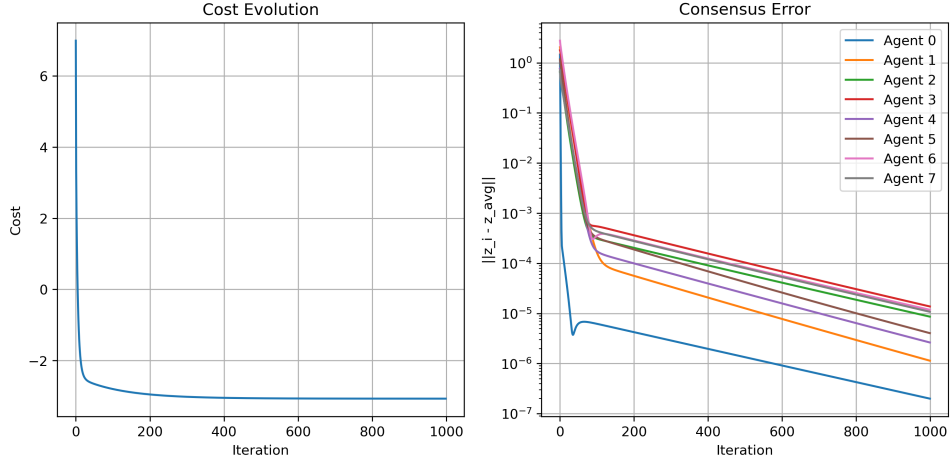


Figure 1.11: Star graph Cost consensus

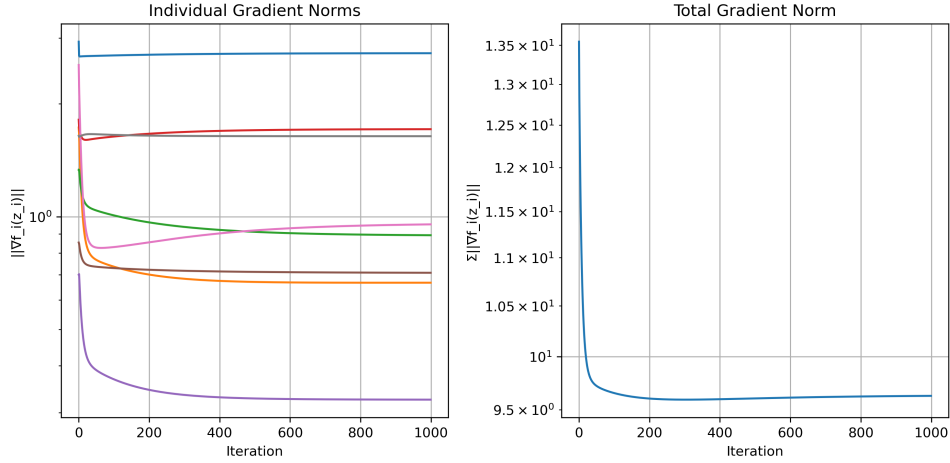


Figure 1.12: Star graph Gradient norms

1.2 Task 1.2: Cooperative Multi-Robot Target Localization

1.2.1 Gradient Tracking implementation

The Gradient Tracking algorithm, previously detailed, is applied to a more realistic scenario where agents cooperate to localize targets within an environment. Each of the N robots has a known position $p_i \in \mathbb{R}^d$ and can perform a noisy distance measurements $d_{i\tau}$ to each of the N_T targets. The final goal is to find the optimal estimates for the target positions, denoted by the decision vector $z = \text{col}(z_1, \dots, z_{N_T}) \in \mathbb{R}^{dN_T}$, where $z_\tau \in \mathbb{R}^d$ is

the estimated position of the τ -th target. This is formulated as a distributed optimization problem aimed at minimizing the sum of local cost functions. To achieve this, the following local cost function is assigned to each agent i :

$$\ell_i(z) := \sum_{\tau=1}^{N_T} (d_{i\tau}^2 - \|z_\tau - p_i\|^2)^2$$

This function penalizes the squared difference between the squared measured noisy distance ($d_{i\tau}^2$) and the squared Euclidean distance from the robot's position p_i to the current target estimate z_τ .

The gradient of the local cost function with respect to a single target estimate is given by:

$$\nabla_{z_\sigma} \ell_i(z) = -4 (d_{i\sigma}^2 - \|z_\sigma - p_i\|^2) (z_\sigma - p_i)$$

The complete gradient $\nabla \ell_i(z)$ is a block vector formed by concatenating the gradients for each target $\tau = 1, \dots, N_T$.

With the cost function and its gradient defined, the agents execute the Gradient Tracking algorithm. Each agent i maintains a local estimate of the target positions $z_i(k)$ and a gradient tracking variable $s_i(k)$ at each iteration k . Through successive iterative updates, the local estimates z_i converge to a common optimal solution z^* , providing the estimated locations of all targets.

Table 1.1: Simulation Parameters for Multi-Robot System

Parameter	Symbol / Variable	Value
Decision dimension	d	2
Number of robots	N	8
Number of targets	N_T	2
Step size	α	10^{-3}
Noise std. deviation	<code>noise_std</code>	0.2

1.2.2 Results

Since from section 1.1 we can see that results are very similar among them when switching type of graph (cycle, star, random, path) for sake of simplicity we are from now on presenting only the random graph case. In this case we will present two examples by simulating a more noisy measurement (increased standard deviation of noise on the measure from 0.2 to 0.4). As it is noticeable from the target estimation error Figure 1.18 having more noise in the measurement the final configuration still reaches consensus but results in higher final error.

Random graph

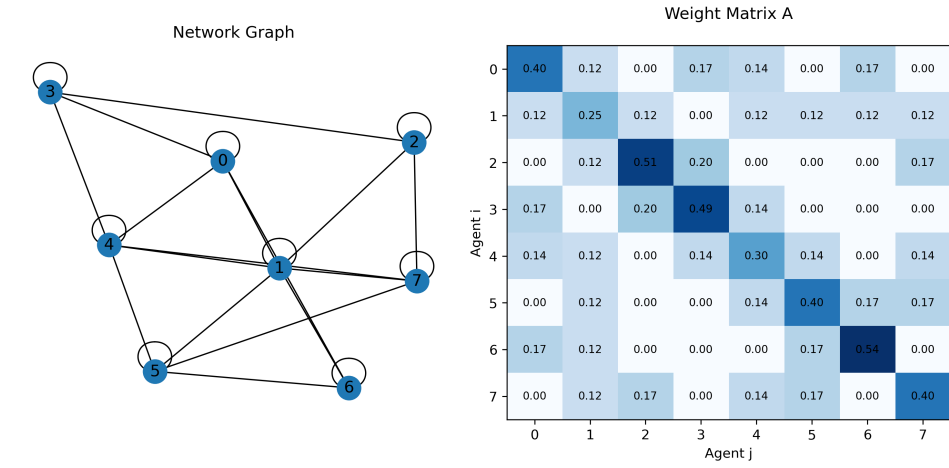


Figure 1.13: Random graph Topology

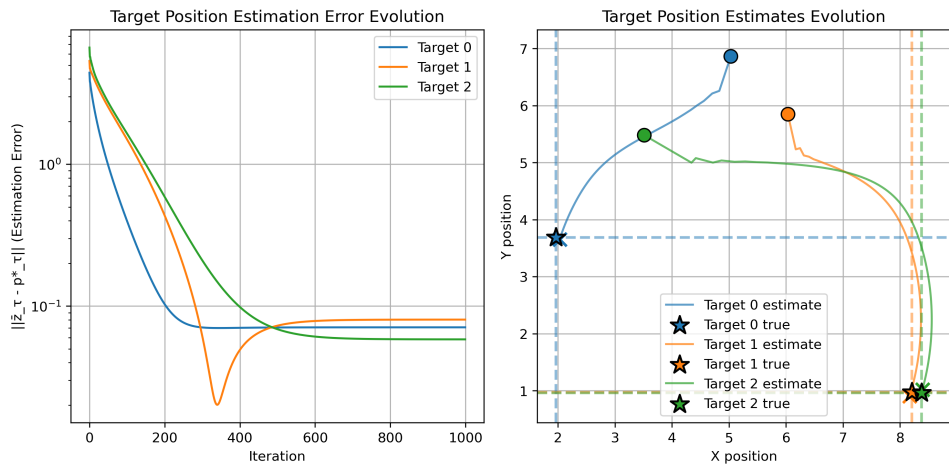


Figure 1.14: Random graph Target estimation

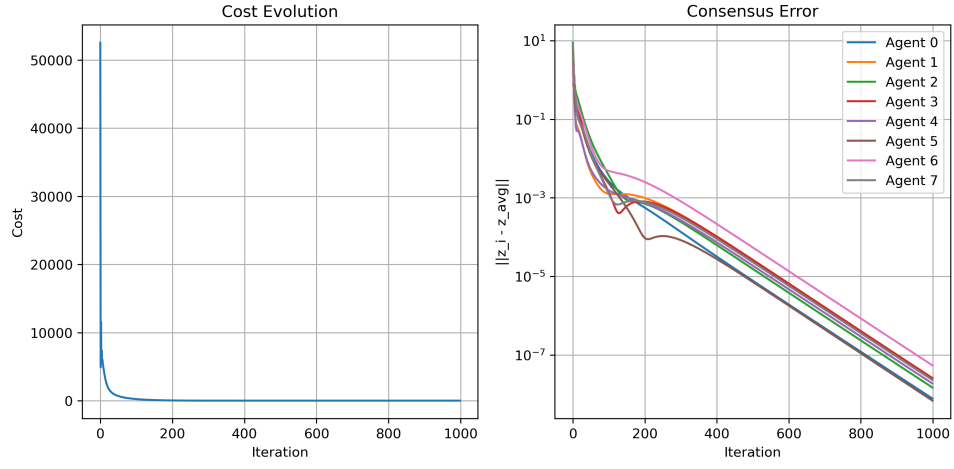


Figure 1.15: Random graph Cost consensus (Task 1.2)

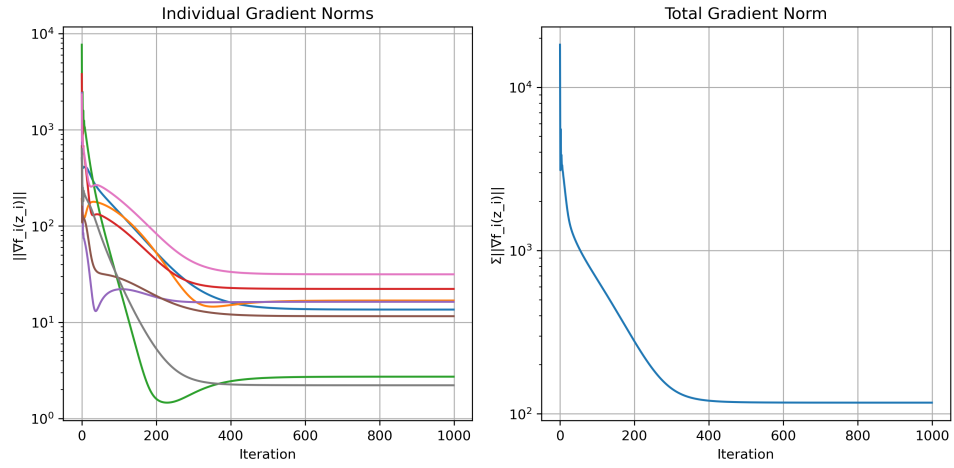


Figure 1.16: Random graph Gradient norms (Task 1.2)

Random graph with standard deviation 0.4

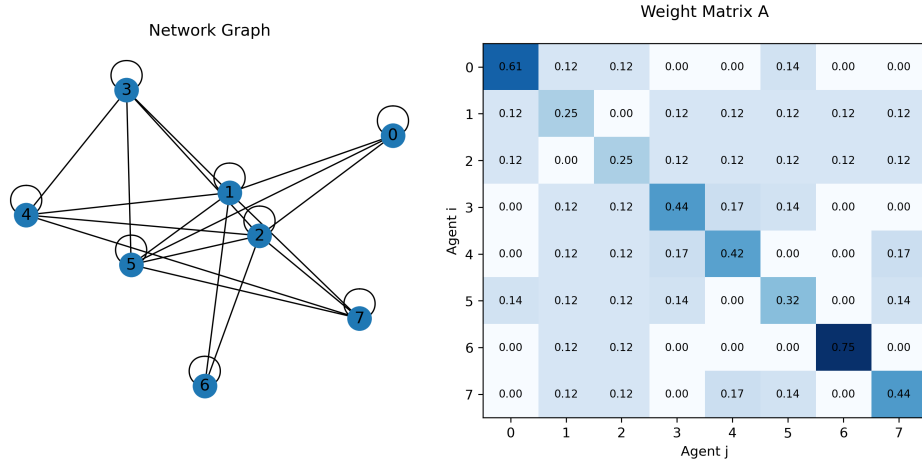


Figure 1.17: Random graph ($\sigma = 0.4$) Topology

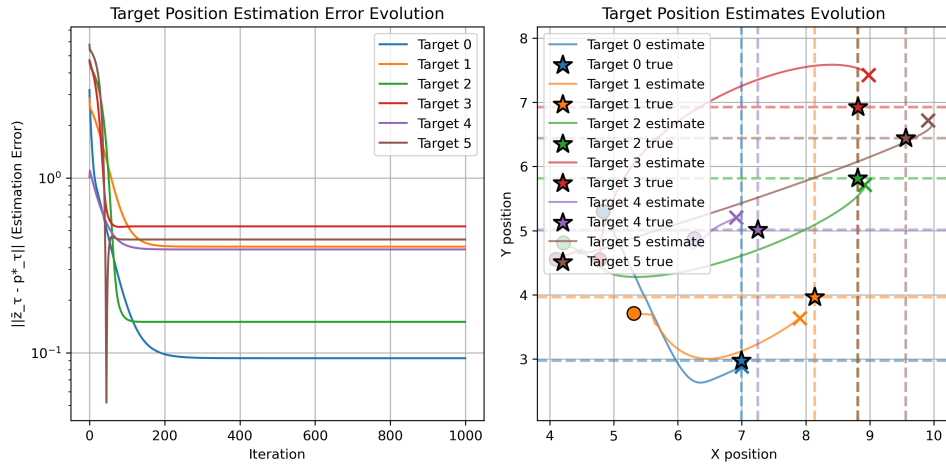
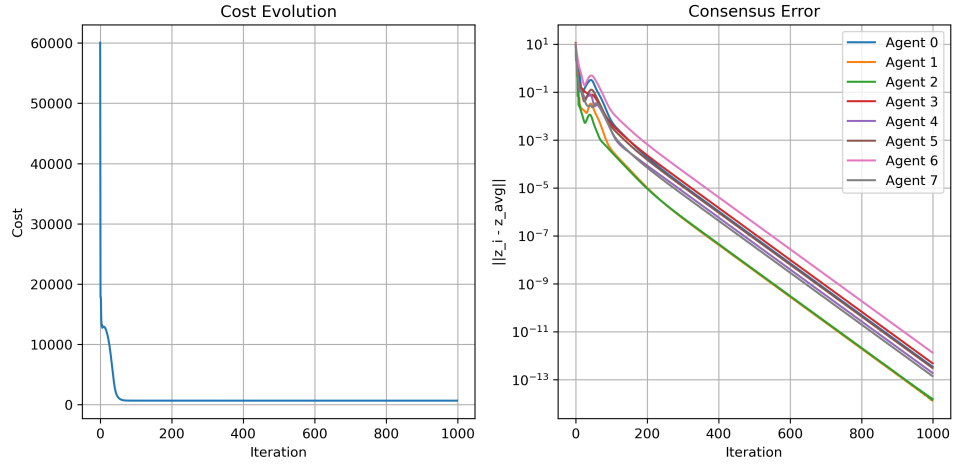
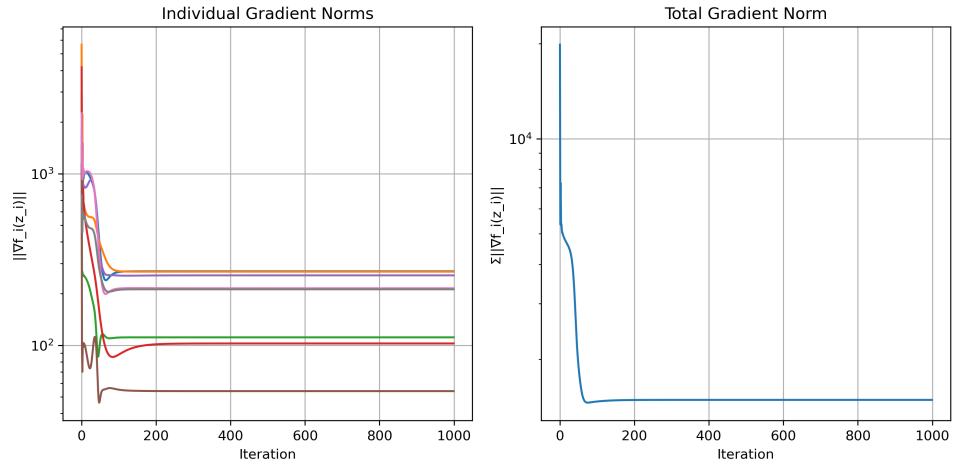


Figure 1.18: Random graph ($\sigma = 0.4$) Target estimation

Figure 1.19: Random graph ($\sigma = 0.4$) Cost consensusFigure 1.20: Random graph ($\sigma = 0.4$) Gradient norms

Chapter 2

Task 2: Aggregative Optimization for Multi-Robot Systems

2.1 Task 2.1: Distributed Aggregative Optimization

2.1.1 Aggregative Tracking algorithm

In this section, we present the distributed strategy adopted to solve the fleet control problem. Unlike the standard consensus optimization in Task 1, this scenario requires minimizing a global cost function that depends not only on the local state of each robot z_i but also on an aggregative variable $\sigma(z)$, which represents the collective behavior of the fleet (in this case, the barycenter).

The optimization problem is formally stated as:

$$\min_{z \in \mathbb{R}^{dN}} \sum_{i=1}^N \ell_i(z_i, \sigma(z)) \quad \text{with} \quad \sigma(z) := \frac{1}{N} \sum_{i=1}^N \phi_i(z_i)$$

where $\phi_i(z_i) = z_i$ for all agents, making $\sigma(z)$ the exact average of the fleet's positions.

To solve this, we employ the *Aggregative Tracking* algorithm. This method extends the Gradient Tracking approach by introducing two dynamic auxiliary variables for each agent:

- s_i^k : tracks the global aggregative variable $\sigma(z^k)$.
- v_i^k : tracks the average of the gradients with respect to the aggregative variable, specifically $\frac{1}{N} \sum_{j=1}^N \nabla_2 \ell_j(z_j^k, \sigma^k)$.

The update laws for the algorithm at iteration k are given by:

$$\begin{aligned} z_i^{k+1} &= z_i^k - \alpha \left(\nabla_1 \ell_i(z_i^k, s_i^k) + \nabla \phi_i(z_i^k)^\top v_i^k \right) \\ s_i^{k+1} &= \sum_{j \in \mathcal{N}_i} a_{ij} s_j^k + \phi_i(z_i^{k+1}) - \phi_i(z_i^k) \\ v_i^{k+1} &= \sum_{j \in \mathcal{N}_i} a_{ij} v_j^k + \nabla_2 \ell_i(z_i^{k+1}, s_i^{k+1}) - \nabla_2 \ell_i(z_i^k, s_i^k) \end{aligned}$$

where $\nabla_1 \ell_i$ and $\nabla_2 \ell_i$ denote the gradients of the cost function with respect to the first argument (z_i) and the second argument (σ), respectively. The initialization is set as $s_i^0 = \phi_i(z_i^0)$ and $v_i^0 = \nabla_2 \ell_i(z_i^0, s_i^0)$. Since $\phi_i(z_i) = z_i$, it follows that $\nabla \phi_i(z_i) = I$, simplifying the descent direction in the first equation.

Cost Function Design and Collision Avoidance

The design of the local cost function ℓ_i is crucial for defining the behavior of the multi-robot system. Our objective is: (1) track a private target r_i , (2) maintain fleet cohesion by staying close to the aggregative variable (barycenter), and (3) ensure safety by avoiding collisions with other agents. To address the safety requirement, we incorporate a Collision Avoidance mechanism based on barrier functions. As the distance between two robots approaches a critical safety threshold distance $\Delta safe$, the cost grows asymptotically to infinity, effectively creating a repulsive force that prevents collision.

The composite local cost function is defined as:

$$\ell_i(z, \sigma) = \gamma \|z_i - r_i\|^2 + (1 - \gamma) \|z_i - \sigma\|^2 + \mu \sum_{j \neq i} B_{ij}(z_i, z_j)$$

where:

- $\gamma \in [0, 1]$ balances the trade-off between tracking the private target r_i and maintaining formation tightness with respect to the barycenter σ .
- $\mu > 0$ is a weighting parameter for the collision avoidance term.
- B_{ij} is the logarithmic barrier function defined as:

$$B_{ij}(z_i, z_j) = -\ln(\|z_i - z_j\|^2 - \Delta safe^2)$$

Here, $\Delta safe$ represents the minimum allowable distance between agents. The term ensures that the cost becomes huge if $\|z_i - z_j\| \rightarrow \Delta safe$, guaranteeing that the gradient descent step pushes the agent away from the boundary.

Consequently, the partial gradients required for the Aggregative Tracking update steps are:

$$\begin{aligned}\nabla_1 \ell_i(z_i, \sigma) &= 2\gamma(z_i - r_i) + 2(1 - \gamma)(z_i - \sigma) + \mu \sum_{j \neq i} \nabla_{z_i} B_{ij} \\ \nabla_2 \ell_i(z_i, \sigma) &= -2(1 - \gamma)(z_i - \sigma)\end{aligned}$$

where the gradient of the barrier term with respect to the agent's position is given by:

$$\nabla_{z_i} B_{ij} = \frac{-2(z_i - z_j)}{\|z_i - z_j\|^2 - \Delta safe^2}$$

This formulation allows the fleet to move coherently while dynamically adjusting individual trajectories to prevent collisions.

2.1.2 Results

In this section, we present the results obtained by testing various combinations of the control parameters μ and γ . The parameter γ regulates the trade-off between the individual objective of reaching a private target and the collective goal of maintaining a compact formation around the fleet's barycenter.

The impact of this parameter is evident when observing the extreme cases. For $\gamma = 1$ in Figure 2.13, the robots focus entirely on tracking their respective targets, effectively ignoring the fleet's configuration. Conversely, as the value of γ decreases, the agents prioritize fleet cohesion. In the limit case of $\gamma = 0$ in imageFigure 2.1, the attractive force toward private targets vanishes, causing all robots to collapse into a single point at the global barycenter.

To prevent such behavior and ensure physical safety, the parameter μ is introduced to weight the barrier function. The effect of this collision avoidance mechanism is clearly visible in Figure Figure 2.4. Here, despite the aggregation force driving the robots toward the barycenter ($\gamma = 0$), the active barrier term ($\mu > 0$) forces them to maintain a safety distance, resulting in a tight but non-colliding formation.

Combination: $\mu = 0.0$ $\gamma = 0.0$

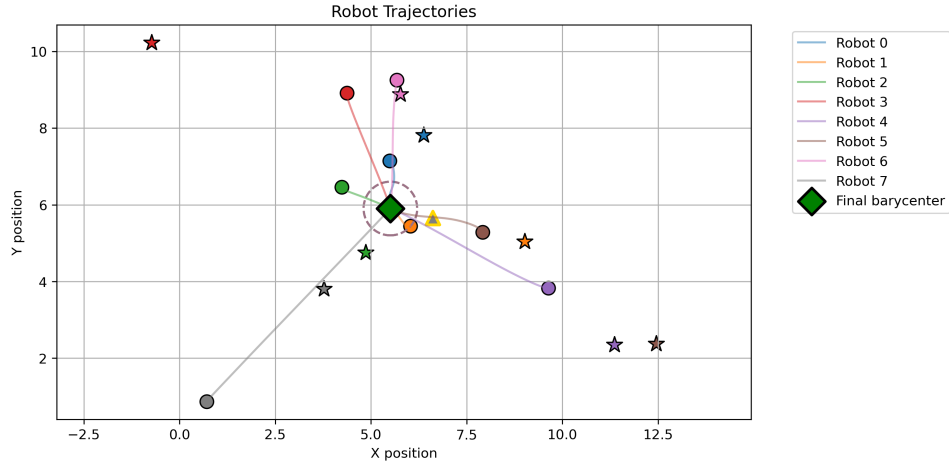


Figure 2.1: Robots trajectories with $\mu = 0.0$ $\gamma = 0.0$

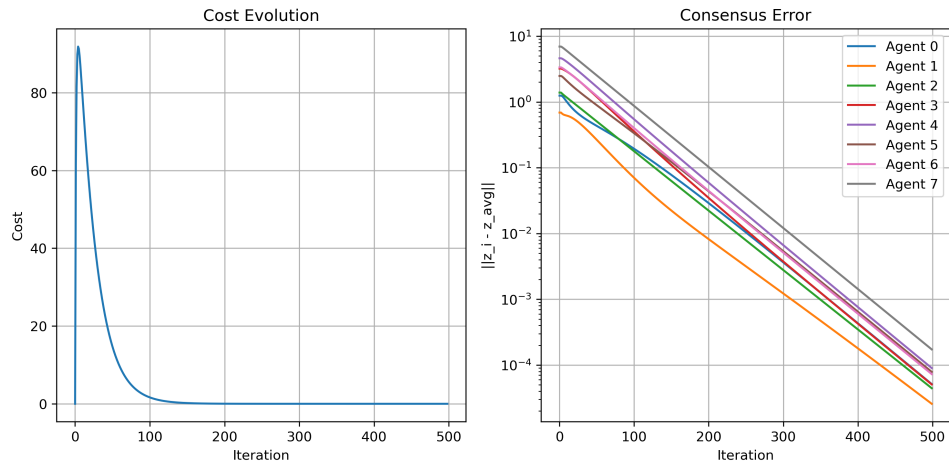


Figure 2.2: Evolution of total cost and consensus errors with $\mu = 0.0$ $\gamma = 0.0$

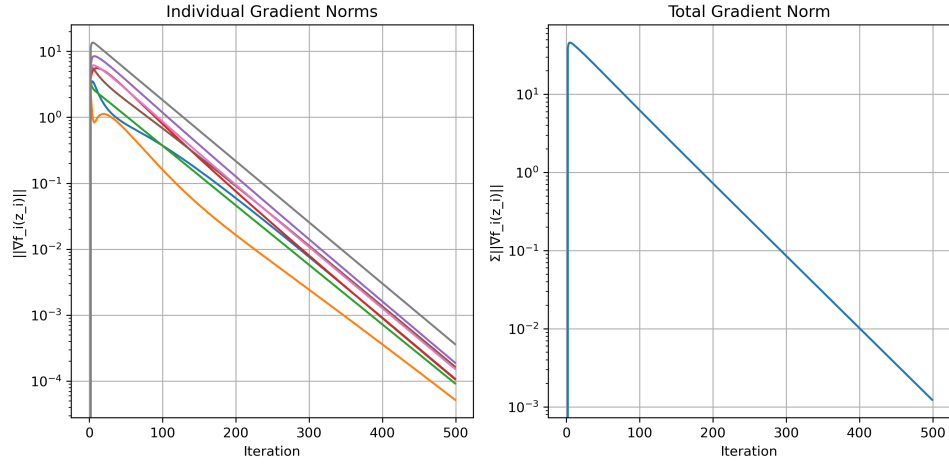


Figure 2.3: Evolution of gradient norms with $\mu = 0.0$ $\gamma = 0.0$

Combination: $\mu = 0.3$ $\gamma = 0.0$

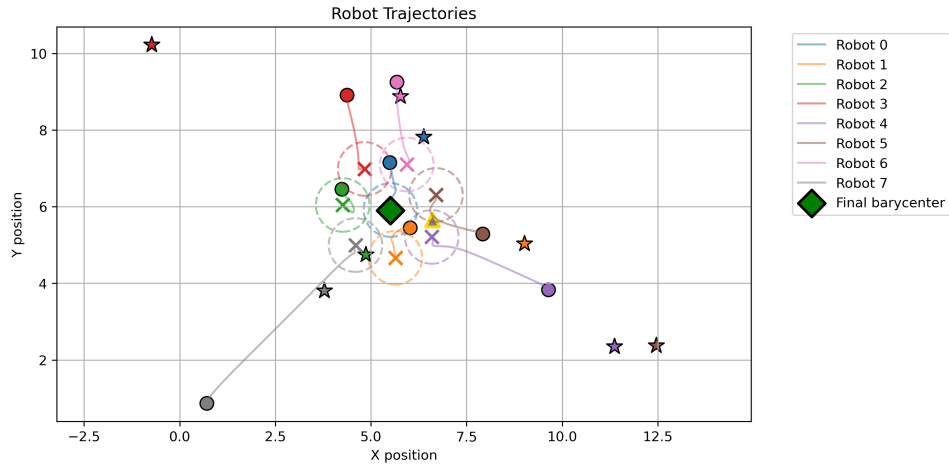


Figure 2.4: Robots trajectories with $\mu = 0.3$ $\gamma = 0.0$

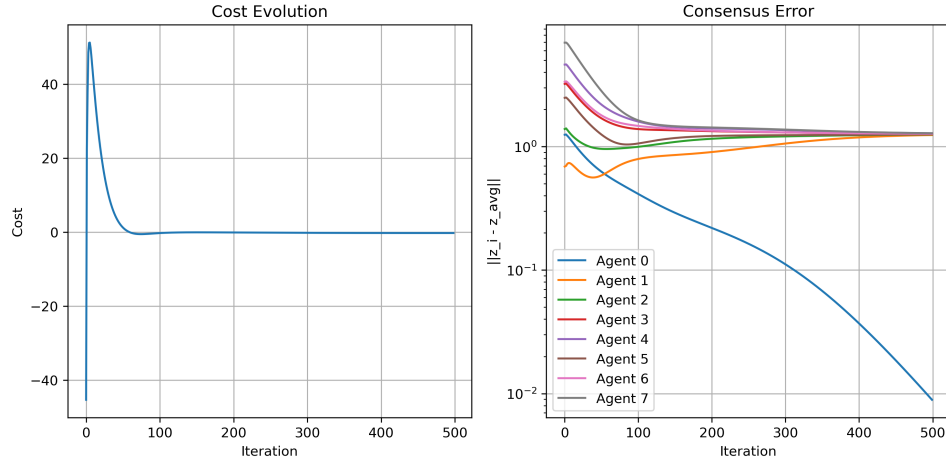


Figure 2.5: Evolution of total cost and consensus errors with $\mu = 0.3$ $\gamma = 0.0$

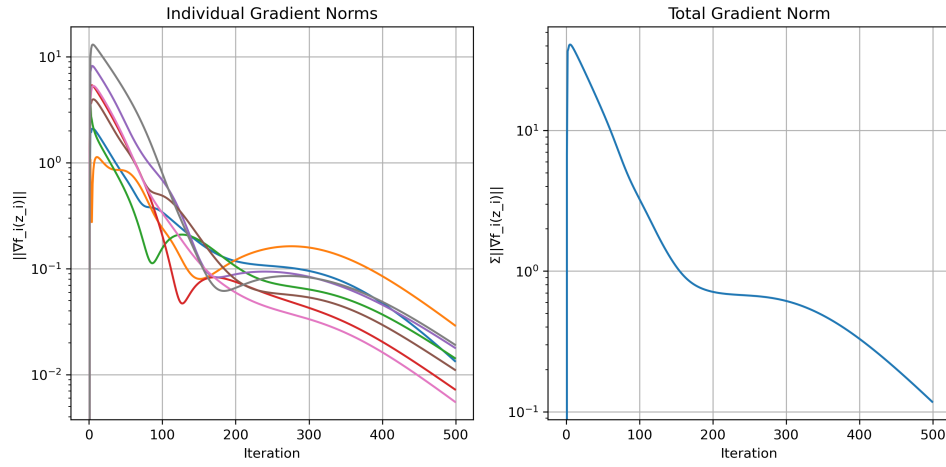


Figure 2.6: Evolution of gradient norms with $\mu = 0.3$ $\gamma = 0.0$

Combination: $\mu = 0.0$ $\gamma = 0.5$

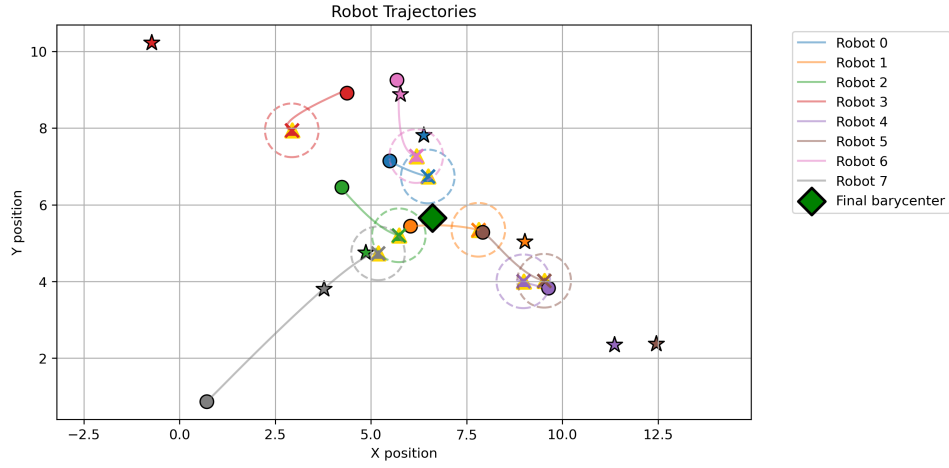


Figure 2.7: Robots trajectories with $\mu = 0.0$ $\gamma = 0.5$

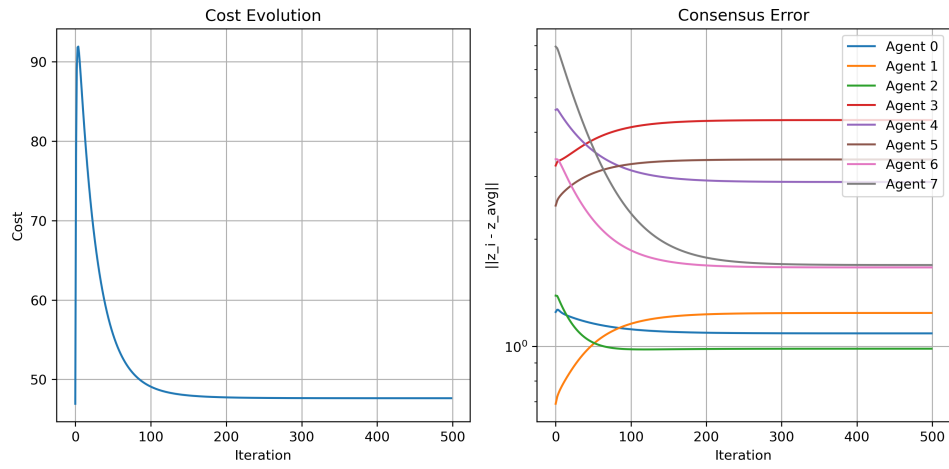


Figure 2.8: Evolution of total cost and consensus errors with $\mu = 0.0$ $\gamma = 0.5$

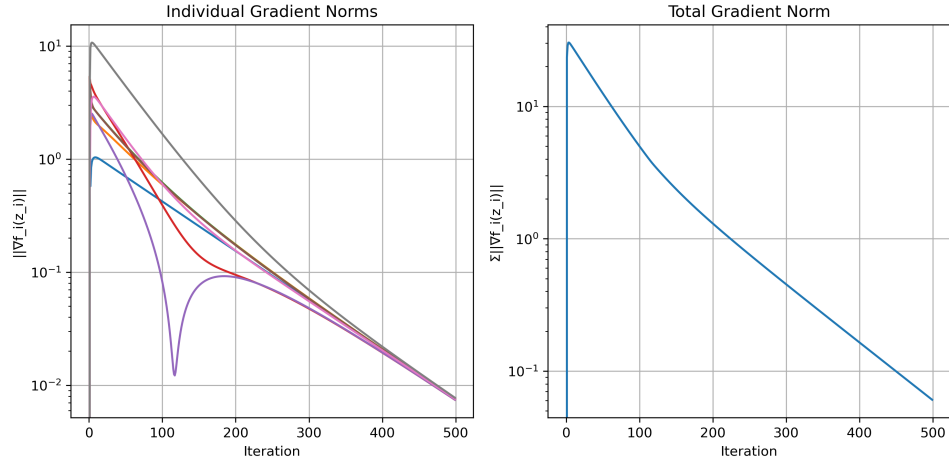


Figure 2.9: Evolution of gradient norms with $\mu = 0.0$ $\gamma = 0.5$

Combination: $\mu = 0.3$ $\gamma = 0.5$

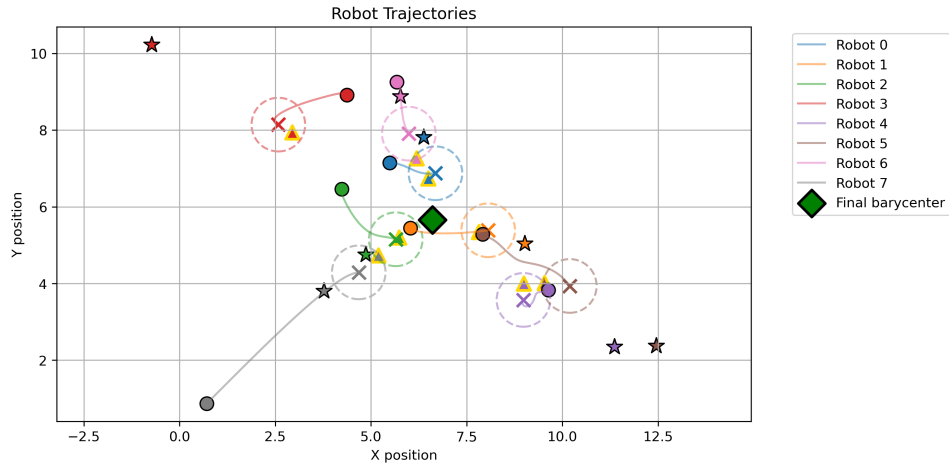


Figure 2.10: Robots trajectories with $\mu = 0.3$ $\gamma = 0.5$

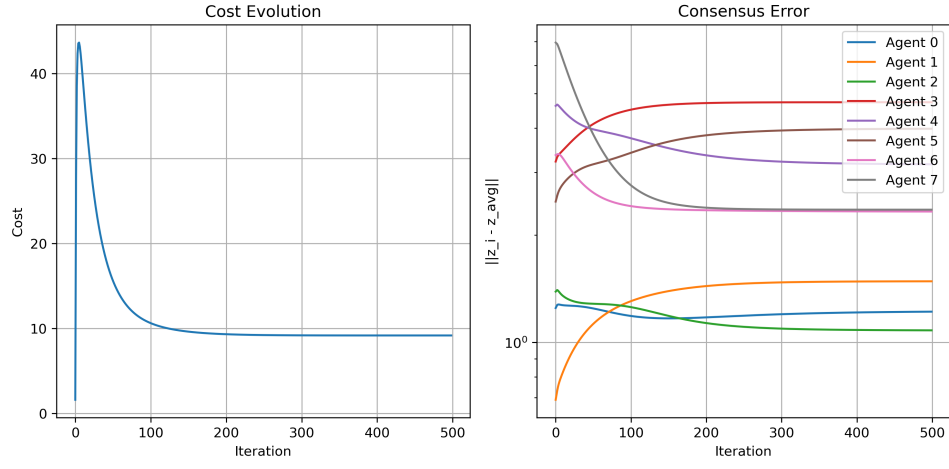


Figure 2.11: Evolution of total cost and consensus errors with $\mu = 0.3$ $\gamma = 0.5$

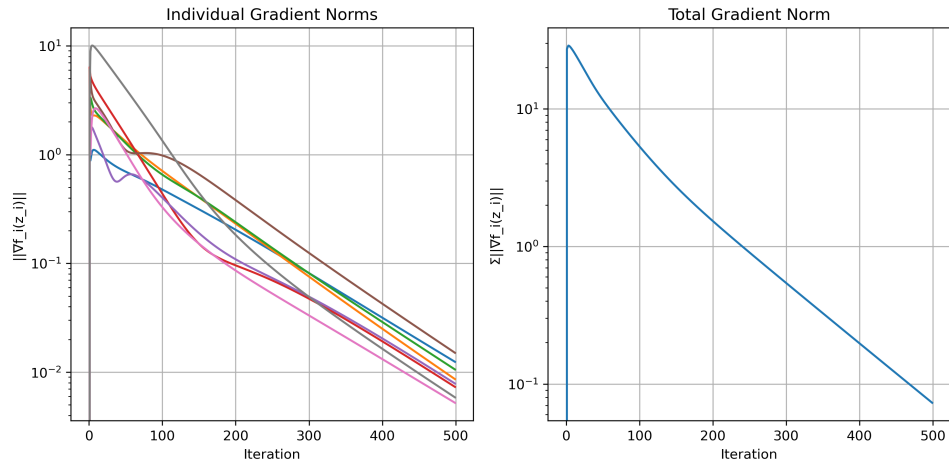


Figure 2.12: Evolution of gradient norms with $\mu = 0.3$ $\gamma = 0.5$

Combination: $\mu = 0.0$ $\gamma = 1.0$

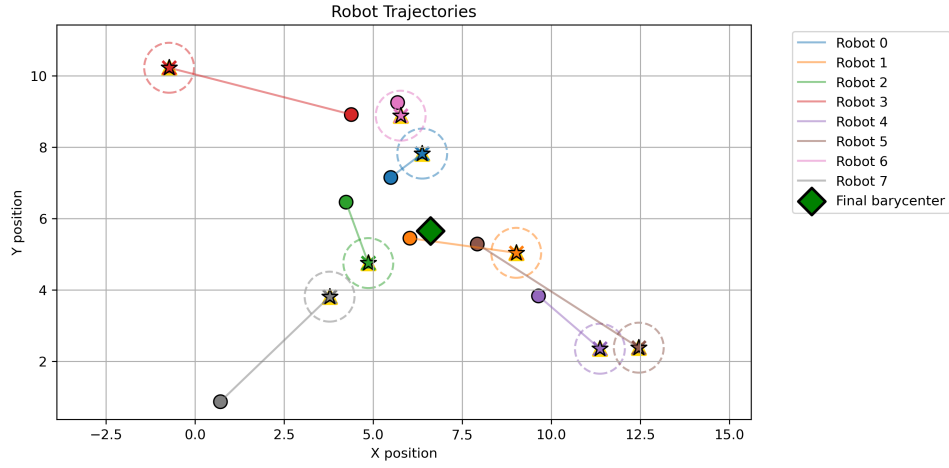


Figure 2.13: Robots trajectories with $\mu = 0.0$ $\gamma = 1.0$

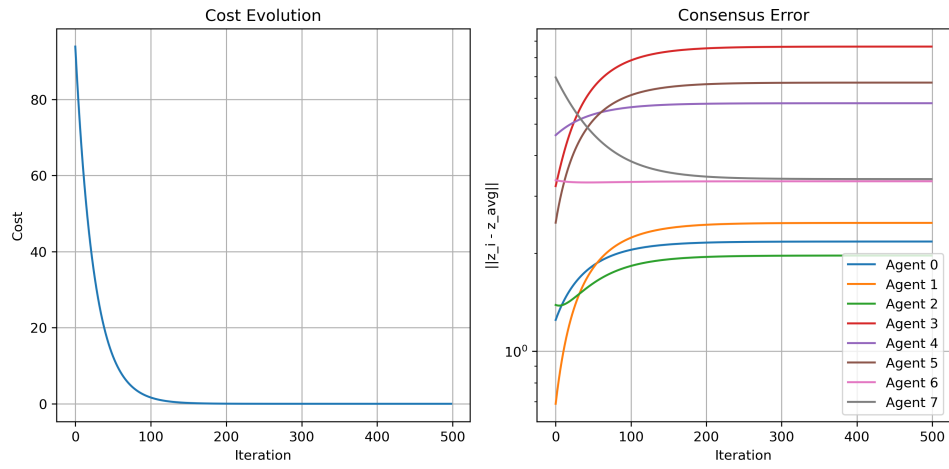


Figure 2.14: Evolution of total cost and consensus errors with $\mu = 0.0$ $\gamma = 1.0$

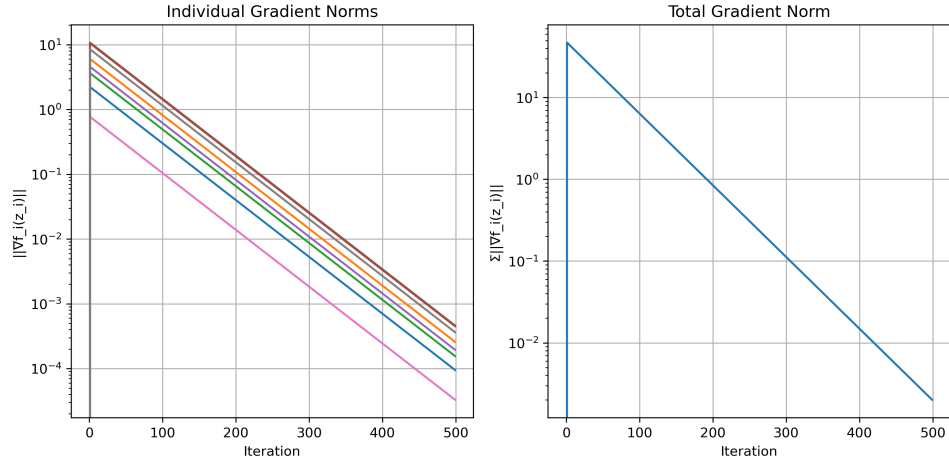


Figure 2.15: Evolution of gradient norms with $\mu = 0.0$ $\gamma = 1.0$

Combination: $\mu = 0.3$ $\gamma = 1.0$

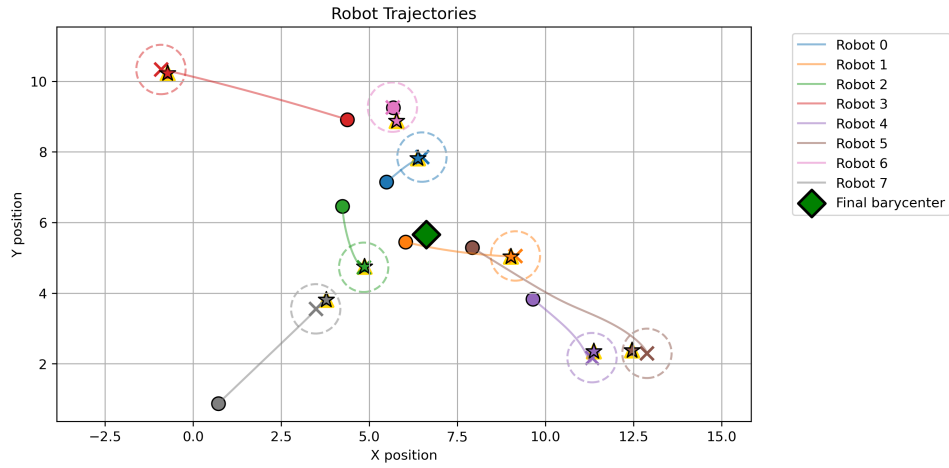


Figure 2.16: Robots trajectories with $\mu = 0.3$ $\gamma = 1.0$

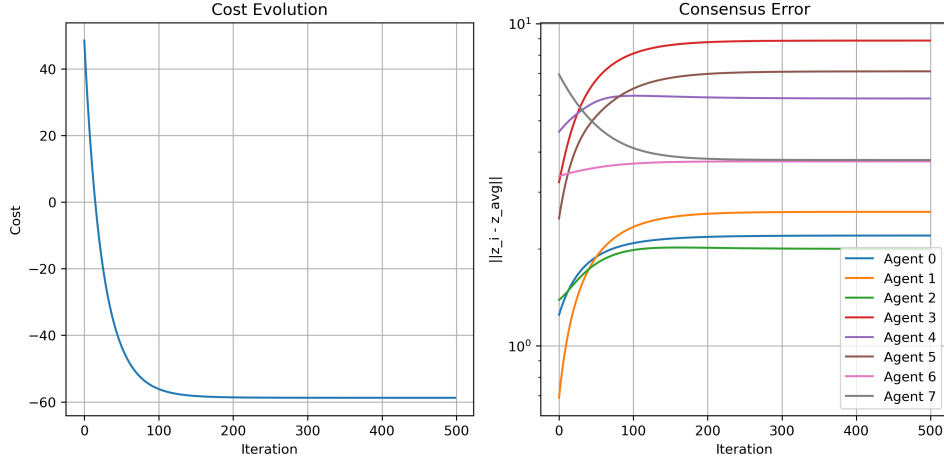


Figure 2.17: Evolution of total cost and consensus errors with $\mu = 0.3$ $\gamma = 1.0$

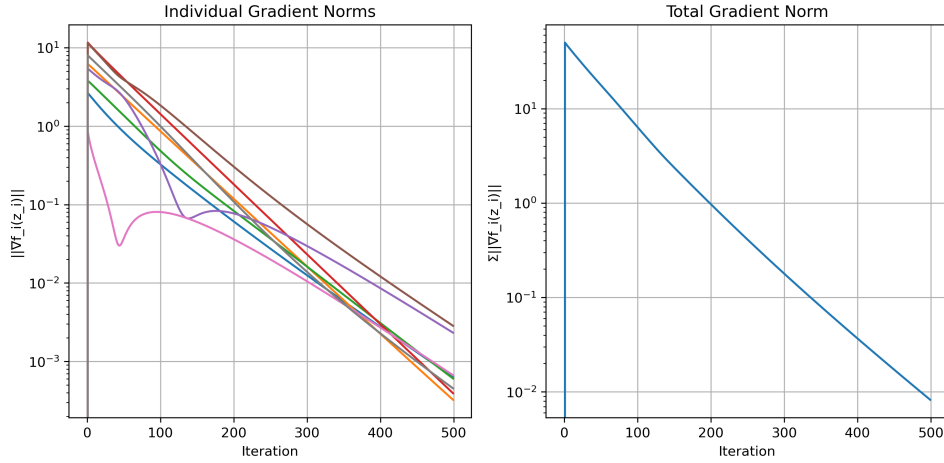


Figure 2.18: Evolution of gradient norms with $\mu = 0.3$ $\gamma = 1.0$

2.2 Task 2.2: ROS 2 Implementation

To validate the practical applicability of the Aggregative Tracking algorithm, we ported the strategy from a centralized Python simulation to a distributed ROS 2 environment. This step allows us to test the algorithm under realistic conditions where agents are distinct processes that must communicate asynchronously. Each robot in the fleet is instantiated as a separate ROS 2 Node. These nodes encapsulate the agent's local state (z_i, s_i, v_i) and execute the optimization logic. They are unaware of the global state and rely solely on local parameters and incoming data. To compute the consensus

terms (e.g., $\sum a_{ij}s_j^k$), the agent must explicitly receive messages from its neighbors. This enforces true decentralization, as agent i cannot access z_j unless agent j actively publishes it.

2.2.1 Results

The results obtained from the ROS 2 implementation align perfectly with the theoretical expectations and the simulations conducted in Task 2.1. The agents exhibit robust cooperative behavior, successfully converging to the optimal solution in all tested scenarios ($\gamma \in \{0.0, 0.5, 1.0\}$).

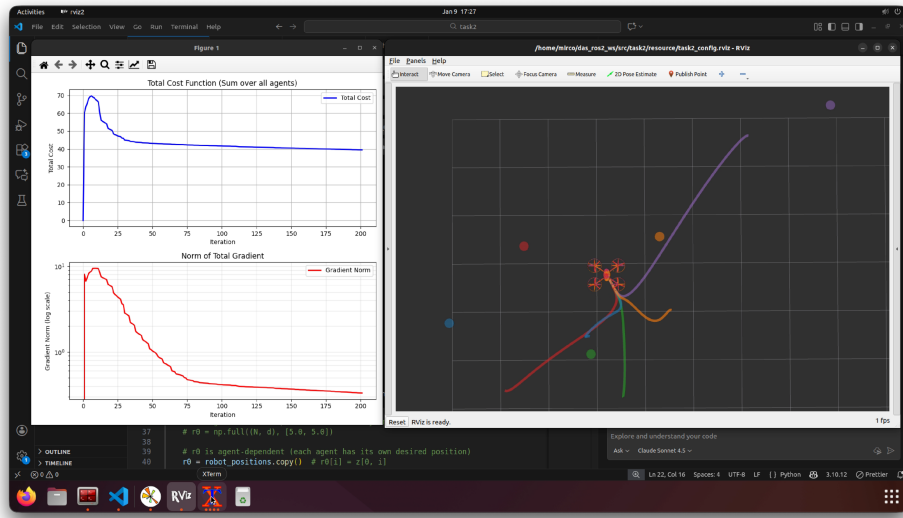


Figure 2.19: Robots trajectories with cost and gradient norm plots: $\gamma = 0.0$

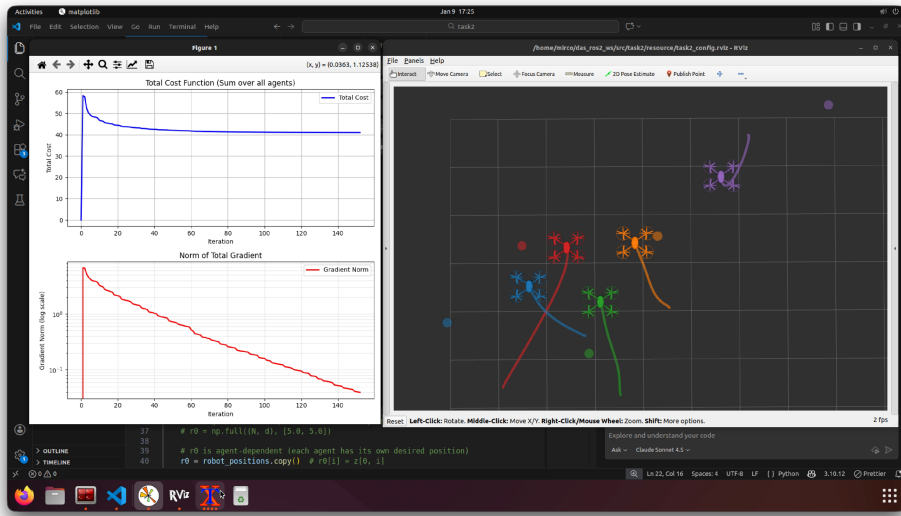


Figure 2.20: Robots trajectories with cost and gradient norm plots: $\gamma = 0.5$

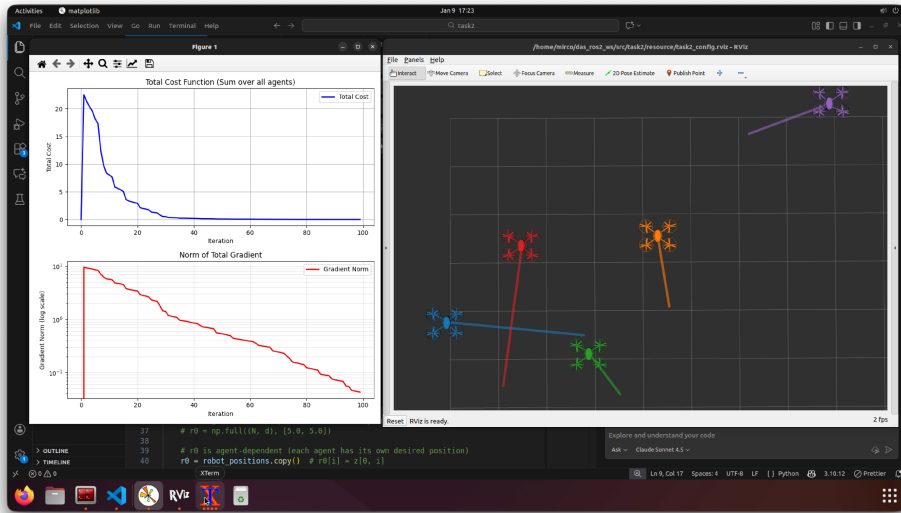


Figure 2.21: Robots trajectories with cost and gradient norm plots: $\gamma = 1.0$

Conclusions

2.3 Conclusion

This project explored the application of distributed optimization techniques to autonomous multi-robot systems, focusing on two distinct operational challenges: Cooperative Target Localization and Aggregative Fleet Control. By implementing and analyzing the Gradient Tracking and Aggregative Tracking algorithms, the work demonstrated how complex global objectives can be achieved through purely local interactions between agents.

In the localization phase (Task chapter 1), it was observed that the Gradient Tracking algorithm provides a robust solution for consensus optimization. Simulations conducted across various network topologies (including cycle, path, star, and random graph) showed a consistent monotonic decrease in the global cost function. The results highlighted that, regardless of the communication structure, the fleet successfully reached stability and consensus on the estimated target positions, verifying the algorithm's theoretical convergence properties.

For the fleet control problem (Task chapter 2), the Aggregative Tracking strategy was implemented to manage the trade-off between tracking individual targets and maintaining fleet cohesion. The experiments demonstrated that the behavior of the system can be effectively tuned via the parameter γ , allowing the fleet to transition smoothly from a loose formation to a tight cluster around the barycenter. Furthermore, the integration of a logarithmic barrier function proved essential for operation, ensuring collision avoidance with only a marginal compromise in the final task accomplishment.

Finally, the transition from a centralized simulation to a distributed ROS 2 environment validated the practical applicability of these strategies. Despite the challenges introduced by asynchronous communication and independent processing nodes, the ROS 2 implementation replicated the optimal trajectories observed in the theoretical simulations.

In summary, this project confirms that distributed optimization strategies

effectively manage the dynamics of autonomous fleets. They offer a scalable and resilient alternative to centralized control, capable of handling real-world constraints such as limited communication and the need for safety-critical collision avoidance.